[UNIVERSIDAD / FACULTAD]
[Carrera]

FinLang

Un lenguaje de dominio específico para finanzas personales y préstamos

Trabajo Parcial (Hito 1)

Analizadores léxico, sintáctico y semántico con ANTLR4

Curso: [Nombre del curso]
Profesor: [Nombre del profesor]
Sección / Semana: [Sección] — Semana 7

Integrantes
[Apellidos, Nombres — Código]
[Apellidos, Nombres — Código]
[Apellidos, Nombres — Código]

Repositorio: [URL de GitHub]

Lima, Perú — 2026

FinLang: DSL para finanzas personales y préstamos

1. Problemática y motivación

En el Perú, comparar y planificar un crédito (vehicular, personal, hipotecario) exige manejar TEA, TEM, TCEA, cuotas constantes y cronogramas de pago. Hoy esto se resuelve con hojas de cálculo o simuladores web que: (i) no distinguen unidades —un 1500 puede ser soles, dólares o un porcentaje, y es fácil sumar PEN con USD sin advertirlo—; (ii) mezclan datos y lógica, lo que vuelve el modelo difícil de revisar y reutilizar; y (iii) no validan reglas del dominio (tasas negativas, plazos no enteros, montos nulos). Armar un modelo propio en una hoja de cálculo es un proceso largo y propenso a errores silenciosos, y los lenguajes de propósito general obligan a programar la matemática financiera desde cero sin prevenir estos errores.

FinLang es un DSL donde *dinero*, *tasa* y *préstamo* son tipos de primera clase: el compilador detecta los errores financieros antes de ejecutar cualquier cálculo. Es importante porque un error de moneda o de tasa en un crédito se traduce directamente en dinero mal calculado.

2. Objetivos

General: diseñar e implementar un compilador para FinLang, un DSL tipado para modelar finanzas personales y préstamos. **Específicos:** (1) definir el léxico (tipos, monedas, porcentajes, palabras clave) e implementarlo con expresiones regulares en ANTLR4; (2) diseñar gramáticas libres de contexto para cada construcción e implementarlas en ANTLR4; (3) implementar un analizador semántico con tabla de símbolos que verifique tipos, monedas y reglas del dominio (el hito pide ≥ 2 errores; se implementaron 10); (4) validar cada etapa con casos de prueba válidos e inválidos. Para soportar finanzas necesitamos: tipos con unidad (`dinero[PEN]`, `tasa`), una construcción `prestamo` y funciones del dominio (`cuota`, `interesTotal`, `convertir`, `cronograma`).

3. Construcciones del lenguaje (5 ejemplos por construcción)

Construcción	Ejemplos (todos verificados en tests/valido_06_construcciones.fin)

Declaración de variables	`num meses = 12;`
	`dinero saldo = 15000 PEN;`
	`tasa tem = 1.5%;`
	`bool meta = falso;`
	`dinero[USD] ahorro;`

Asignación	`meses = meses + 1;`
	`saldo = saldo + 500 PEN;`
	`tem = 2%;`
	`meta = verdadero;`
	`ahorro = 0 USD;`

Expresiones	`saldo * tem`
	`(saldo - 500 PEN) / 2`
	`saldo > 1000 PEN && meses < 36`
	`!meta || meses > 12`
	`convertir(saldo, USD, 0.27)`

Sentencias selectivas	`si (meta) mostrar "ok";`
	`si (saldo > 0 PEN) { mostrar "con saldo"; }`
	`si (meta) { n = 1; } sino { n = 2; }`
	`si (!meta || meses > 12) mostrar "revisar";`
	`si (meta) n = 1; sino si (meses > 6) n = 2;`

Sentencias iterativas	`mientras (saldo > 0 PEN) { saldo = saldo - 100 PEN; }`
	`mientras (!meta && meses < 36) meses = meses + 1;`
	`para i desde 1 hasta 12 { saldo = saldo + saldo * tem; }`
	`para k desde 1 hasta meses mostrar k;`
	`para j desde 0 hasta meses - 1 mostrar j;`

Préstamos y reportes (dominio)	`prestamo auto { monto: 30000 USD; tea: 12%; plazo: 36; }`
	`dinero cuota1 = cuota(auto);`
	`mostrar "Interés:", interesTotal(auto);`
	`cronograma auto;`
	`dinero[PEN] enSoles = convertir(cuota(auto), PEN, 3.75);`

4. Analizador léxico

Formato TOKEN: lista de lexemas. Las palabras clave se declaran antes que ID para tener prioridad; cualquier otro carácter produce un error léxico [LEX].

Token	Lexemas / expresión regular
`TNUM, TDINERO, TTASA, TBOOL`	num, dinero, tasa, bool
`Palabras clave`	prestamo, monto, tea, plazo, si, sino, mientras, para, desde, hasta, mostrar, cronograma, cuota, interesTotal, convertir
`TRUE, FALSE`	verdadero, falso
`CURRENCY`	PEN, USD, EUR
`NUMERO`	0, 12, 3.75, 15000 — [0-9]+('.'[0-9]+)?
`PORCENTAJE`	18%, 1.5%, 0.25% — [0-9]+('.'[0-9]+)?'%'
`STRING`	"Cuota:", "ok" — '"' ~["\r\n]* '"'
`ID`	saldo, cuotaMensual, ahorro_1 — [a-zA-Z_][a-zA-Z_0-9]*

Token	Lexemas / expresión regular
PLUS, MINUS, MUL, DIV	+, -, *, /
LT, LE, GT, GE, EQ, NEQ	<, <=, >, >=, ==, !=
AND, OR, NOT, ASSIGN	&&, ||, !, =
Delimitadores	( ) { } [ ] ; , : (LPAREN, RPAREN, LBRACE, RBRACE, LBRACK, RBRACK, SEMI, COMMA, COLON)
Ignorados (skip)	espacios, saltos de línea, // comentario, /* comentario */
ERROR_CHAR	cualquier otro carácter (p. ej. #, @, $) → error léxico

5. Analizador sintáctico

Notación: no terminales en camelCase; terminales entre comillas simples o en MAYÚSCULAS (clases de token); | separa alternativas; ε es la cadena vacía. Es una CFG en BNF pura (sin *, ? ni paréntesis): las repeticiones se expresan con recursión (stmtList, exprTail) y la precedencia con niveles (expr → andE → eqE → relE → addE → mulE → unE → prim). En ANTLR4 se implementa con EBNF y recursión izquierda directa (FinLang.g4).

```
// Programa y sentencias
program → stmtList EOF
stmtList → statement stmtList | ε
statement → declaration | assignment | loanDecl | printStmt | ifStmt | whileStmt | forStmt | scheduleStmt | block
block → '{' stmtList '}'
// Declaraciones
declaration → tipo ID init ';'
init → '=' expr | ε
tipo → 'num' | 'dinero' moneda | 'tasa' | 'bool'
moneda → '[' CURRENCY ']' | ε
// Asignaciones
assignment → ID '=' expr ';'
// Expresiones
expr → expr '||' andE | andE
andE → andE '&&' eqE | eqE
eqE → eqE '==' relE | eqE '!=' relE | relE
relE → relE '<' addE | relE '<=' addE | relE '>' addE | relE '>=' addE | addE
addE → addE '+' mulE | addE '-' mulE | mulE
mulE → mulE '*' unE | mulE '/' unE | unE
unE → '-' unE | '!' unE | prim
prim → '(' expr ')' | NUMERO CURRENCY | NUMERO | PORCENTAJE | 'verdadero' | 'falso' | STRING | ID | 'cuota' '(' ID
')' | 'interesTotal' '(' ID ')' | 'convertir' '(' expr ',' CURRENCY ',' expr ')'
// Selectivas
ifStmt → 'si' '(' expr ')' statement elsePart
elsePart → 'sino' statement | ε
// Iterativas
whileStmt → 'mientras' '(' expr ')' statement
forStmt → 'para' ID 'desde' expr 'hasta' expr statement
// Préstamos y reportes
loanDecl → 'prestamo' ID '{' 'monto' ':' expr ';' 'tea' ':' expr ';' 'plazo' ':' expr ';' '}'
printStmt → 'mostrar' expr exprTail ';'
exprTail → ',' expr exprTail | ε
scheduleStmt → 'cronograma' ID ';'
```

Derivaciones más a la izquierda (4 por gramática)

Cada ⇒ aplica una producción sobre el no terminal más a la izquierda; ⇒* agrupa una cadena de producciones unitarias (A → B). Los terminales ya generados se escriben con su lexema. Las derivaciones completas, sin abreviar, están en docs/Derivaciones_completas.md (generadas con tools/derivaciones.py).

Declaraciones

1. `num meses = 12;`

```
declaration
⇒  tipo ID init ;
⇒  num meses init ;
⇒  num meses = expr ;
⇒* num meses = 12 ;
```

2. `dinero saldo = 15000 PEN;`

```
declaration
⇒  tipo ID init ;
⇒  dinero moneda ID init ;
⇒  dinero saldo init ;
⇒  dinero saldo = expr ;
⇒* dinero saldo = 15000 PEN ;
```

3. `dinero[USD] ahorro;`

```
declaration
⇒  tipo ID init ;
⇒  dinero moneda ID init ;
⇒  dinero [ USD ] ahorro init ;
⇒  dinero [ USD ] ahorro ;
```

4. `bool meta = falso;`

```
declaration
⇒  tipo ID init ;
⇒  bool meta init ;
⇒  bool meta = expr ;
⇒* bool meta = falso ;
```

Asignaciones

1. meses = meses + 1;

```
assignment
⇒  meses = expr ;
⇒* meses = addE + mulE ;
⇒* meses = meses + mulE ;
⇒* meses = meses + 1 ;
```

2. tem = 2%;

```
assignment
⇒  tem = expr ;
⇒* tem = 2% ;
```

3. meta = verdadero;

```
assignment
⇒  meta = expr ;
⇒* meta = verdadero ;
```

4. ahorro = 0 USD;

```
assignment
⇒  ahorro = expr ;
⇒* ahorro = 0 USD ;
```

Expresiones

1. saldo * tem

```
expr
⇒* mulE * unE
⇒* saldo * unE
⇒* saldo * tem
```

2. (a + b) / 2

```
expr
⇒* mulE / unE
⇒* ( expr ) / unE
⇒* ( addE + mulE ) / unE
⇒* ( a + mulE ) / unE
⇒* ( a + b ) / unE
⇒* ( a + b ) / 2
```

3. a > 1000 PEN && mes < 36

```
expr
⇒* andE && eqE
⇒* relE > addE && eqE
⇒* a > addE && eqE
⇒* a > 1000 PEN && eqE
⇒* a > 1000 PEN && relE < addE
⇒* a > 1000 PEN && mes < addE
⇒* a > 1000 PEN && mes < 36
```

4. convertir(saldo, USD, 3.75)

```
expr
⇒* convertir ( expr , CURRENCY , expr )
⇒* convertir ( saldo , USD , expr )
⇒* convertir ( saldo , USD , 3.75 )
```

Sentencias selectivas

1. si (meta) mostrar "ok";

```
ifStmt
⇒  si ( expr ) statement elsePart
⇒* si ( meta ) statement elsePart
⇒* si ( meta ) mostrar expr exprTail ; elsePart
⇒* si ( meta ) mostrar "ok" exprTail ; elsePart
⇒  si ( meta ) mostrar "ok" ; elsePart
⇒  si ( meta ) mostrar "ok" ;
```

2. si (a == b) n = 1; sino n = 2;

```
ifStmt
⇒  si ( expr ) statement elsePart
⇒* si ( eqE == relE ) statement elsePart
⇒* si ( a == relE ) statement elsePart
⇒* si ( a == b ) statement elsePart
⇒* si ( a == b ) n = expr ; elsePart
⇒* si ( a == b ) n = 1 ; elsePart
⇒  si ( a == b ) n = 1 ; sino statement
⇒* si ( a == b ) n = 1 ; sino n = expr ;
⇒* si ( a == b ) n = 1 ; sino n = 2 ;
```

3. si (saldo > 0 PEN) { mostrar "ok"; }

```
ifStmt
⇒  si ( expr ) statement elsePart
⇒* si ( relE > addE ) statement elsePart
⇒* si ( saldo > addE ) statement elsePart
⇒* si ( saldo > 0 PEN ) statement elsePart
⇒* si ( saldo > 0 PEN ) { stmtList } elsePart
⇒  si ( saldo > 0 PEN ) { statement stmtList } elsePart
⇒* si ( saldo > 0 PEN ) { mostrar expr exprTail ; stmtList }
elsePart
⇒* si ( saldo > 0 PEN ) { mostrar "ok" exprTail ; stmtList }
elsePart
⇒  si ( saldo > 0 PEN ) { mostrar "ok" ; stmtList } elsePart
⇒  si ( saldo > 0 PEN ) { mostrar "ok" ; } elsePart
⇒  si ( saldo > 0 PEN ) { mostrar "ok" ; }
```

4. si (meta) n = 1; sino si (a > b) n = 2;

```
ifStmt
⇒  si ( expr ) statement elsePart
⇒* si ( meta ) statement elsePart
⇒* si ( meta ) n = expr ; elsePart
⇒* si ( meta ) n = 1 ; elsePart
⇒  si ( meta ) n = 1 ; sino statement
⇒* si ( meta ) n = 1 ; sino si ( expr ) statement elsePart
⇒* si ( meta ) n = 1 ; sino si ( relE > addE ) statement
elsePart
⇒* si ( meta ) n = 1 ; sino si ( a > addE ) statement
elsePart
⇒* si ( meta ) n = 1 ; sino si ( a > b ) statement elsePart
⇒* si ( meta ) n = 1 ; sino si ( a > b ) n = expr ; elsePart
⇒* si ( meta ) n = 1 ; sino si ( a > b ) n = 2 ; elsePart
⇒  si ( meta ) n = 1 ; sino si ( a > b ) n = 2 ;
```

Sentencias iterativas

1. mientras (saldo > 0 PEN) saldo = saldo - cuota1;

```
whileStmt
⇒  mientras ( expr ) statement
⇒* mientras ( relE > addE ) statement
⇒* mientras ( saldo > addE ) statement
⇒* mientras ( saldo > 0 PEN ) statement
⇒* mientras ( saldo > 0 PEN ) saldo = expr ;
⇒* mientras ( saldo > 0 PEN ) saldo = addE - mulE ;
⇒* mientras ( saldo > 0 PEN ) saldo = saldo - mulE ;
⇒* mientras ( saldo > 0 PEN ) saldo = saldo - cuota1 ;
```

2. mientras (!meta && mes < 36) mes = mes + 1;

```
whileStmt
⇒  mientras ( expr ) statement
⇒* mientras ( andE && eqE ) statement
⇒* mientras ( ! unE && eqE ) statement
⇒* mientras ( ! meta && eqE ) statement
⇒* mientras ( ! meta && relE < addE ) statement
⇒* mientras ( ! meta && mes < addE ) statement
⇒* mientras ( ! meta && mes < 36 ) statement
⇒* mientras ( ! meta && mes < 36 ) mes = expr ;
⇒* mientras ( ! meta && mes < 36 ) mes = addE + mulE ;
⇒* mientras ( ! meta && mes < 36 ) mes = mes + mulE ;
⇒* mientras ( ! meta && mes < 36 ) mes = mes + 1 ;
```

3. para i desde 1 hasta 12 saldo = saldo * tem;

```
forStmt
⇒  para i desde expr hasta expr statement
⇒* para i desde 1 hasta expr statement
⇒* para i desde 1 hasta 12 statement
⇒* para i desde 1 hasta 12 saldo = expr ;
⇒* para i desde 1 hasta 12 saldo = mulE * unE ;
⇒* para i desde 1 hasta 12 saldo = saldo * unE ;
⇒* para i desde 1 hasta 12 saldo = saldo * tem ;
```

4. para k desde 1 hasta meses { n = n + 1; }

```
forStmt
⇒  para k desde expr hasta expr statement
⇒* para k desde 1 hasta expr statement
⇒* para k desde 1 hasta meses statement
⇒* para k desde 1 hasta meses { stmtList }
⇒  para k desde 1 hasta meses { statement stmtList }
⇒* para k desde 1 hasta meses { n = expr ; stmtList }
⇒* para k desde 1 hasta meses { n = addE + mulE ; stmtList }
⇒* para k desde 1 hasta meses { n = n + mulE ; stmtList }
⇒* para k desde 1 hasta meses { n = n + 1 ; stmtList }
⇒  para k desde 1 hasta meses { n = n + 1 ; }
```

Préstamos y reportes

1. prestamo p { monto: 1000 PEN; tea: 10%; plazo: 12; }

```
loanDecl
⇒  prestamo p { monto : expr ; tea : expr ; plazo : expr ; }
⇒* prestamo p { monto : 1000 PEN ; tea : expr ; plazo :
expr ; }
⇒* prestamo p { monto : 1000 PEN ; tea : 10% ; plazo : expr ;
}
⇒* prestamo p { monto : 1000 PEN ; tea : 10% ; plazo : 12 ; }
```

2. mostrar "Cuota:", cuota(auto);

```
printStmt
⇒  mostrar expr exprTail ;
⇒* mostrar "Cuota:" exprTail ;
⇒  mostrar "Cuota:" , expr exprTail ;
⇒* mostrar "Cuota:" , cuota ( auto ) exprTail ;
⇒  mostrar "Cuota:" , cuota ( auto ) ;
```

3. cronograma auto;

```
scheduleStmt
⇒  cronograma auto ;
```

4. mostrar interesTotal(auto);

```
printStmt
⇒  mostrar expr exprTail ;
⇒* mostrar interesTotal ( auto ) exprTail ;
⇒  mostrar interesTotal ( auto ) ;
```

6. Analizador semántico

Se usa un visitor sobre el árbol de ANTLR4 con una pila de tablas de símbolos (un ámbito por bloque) y un sistema de tipos num, tasa, bool, texto, dinero[MONEDA] y prestamo. Cada error indica línea y columna; una expresión con error devuelve el tipo error, que evita errores en cascada.

Cód.	Error semántico	Ejemplo
E01	Variable no declarada	`saldoFinal = 1;  (sin declarar)`
E02	Identificador duplicado en el ámbito	`num n = 1; num n = 2;`
E03	Tipos incompatibles	`tasa t = 10%; t = 5 PEN;`
E04	Mezcla de monedas sin conversión	`100 PEN + 20 USD  (usar convertir)`
E05	Operación inválida para los tipos	`dinero a = 100 PEN; a + 5`
E06	Tasa negativa o fuera de (0%, 100%]	`tea: -5%;  tea: 150%;`
E07	Monto o plazo inválido	`monto: -1000 PEN;  plazo: 2.5;`
E08	Condición no booleana	`num n = 3; si (n) { ... }`
E09	Identificador que no es préstamo	`num n = 1; cronograma n;`
E10	dinero sin moneda	`dinero ahorro;`

7. Pruebas y resultados

La carpeta tests/ tiene 23 casos (6 válidos y 17 inválidos: uno por cada error E01–E10, variantes de tasa/monto negativos, ámbito, un caso léxico, uno sintáctico y uno con varios errores a la vez). Cada archivo declara su resultado esperado (// ESPERA: E04) y python run_tests.py los ejecuta: resultado actual 23 OK, 0 FAIL. Ejemplo de salida (tests/err_multiple.fin):

```
[E06] Línea 2:40 - la TEA -12% no es válida (tasa negativa); debe estar en (0%, 100%]
[E04] Línea 3:16 - no se pueden mezclar monedas distintas (PEN y USD); use convertir()
[E08] Línea 5:5 - la condición debe ser bool, se encontró num
[E01] Línea 6:9 - 'saldoFinal' no ha sido declarado
4 error(es) encontrado(s).
```

Etapa 2 (planificada): cálculo real de cuotas, cronograma y TCEA (con comisiones y seguros), y generación de código con LLVM.